

Experiment 2: Client–Server Communication Between Two Virtual Machines

Aim

To configure two virtual machine instances in VirtualBox to establish client–server communication, and to implement a basic TCP-based chat application between the two virtual machines.

Objectives

- Create/prepare two Ubuntu Virtual Machines in VirtualBox.
- Configure both VMs on the same virtual network.
- Assign/check the IP address of both VMs.
- Test connectivity using the ping command.
- Designate one VM as the Server.
- Designate the other VM as the Client.
- Create a TCP server program on the Server VM.
- Create a TCP client program on the Client VM.
- Establish a connection between the two VMs.
- Send and receive messages through the chat application.
- Demonstrate the working chat application.
- Capture screenshots of configuration, connectivity and chat communication.

Step 1: Create/Prepare Two Ubuntu Virtual Machines

Two Ubuntu Virtual Machines were created and installed in VirtualBox: VM1 (hostname sk-naser-hussain-VirtualBox, network interface MAC address ending c8:c6:c8) and VM2 (hostname sk-naser-hussain-VirtualBox, network interface MAC address ending 0f:36:1b). Both machines are connected using an internal/host-only network adapter so that they can communicate with each other over a common subnet.

Step 2: Configure Both VMs on the Same Virtual Network & Assign IP Addresses

The 'ip addr' command was first used on VM1 to check the existing network configuration. As shown below, the interface enp0s3 was up but had not yet been assigned an IPv4 address.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:c6:c8 brd ff:ff:ff:ff:ff:ff
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig 1. Initial network interface status on VM1 (enp0s3 up, no IPv4 address assigned).

A static IP address was then assigned to VM1 using the command 'sudo ip addr add 192.168.10.1/24 dev enp0s3', placing VM1 on the 192.168.10.0/24 subnet.

```

sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:c6:c8 brd ff:ff:ff:ff:ff:ff
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo ip addr add 192.168.10.1/24 dev enp0s3
[sudo] password for sk-naser-hussain:
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$

```

Fig 2. Assigning static IP address 192.168.10.1/24 to VM1 (enp0s3).

The same procedure was repeated on VM2, assigning it the IP address 192.168.10.2/24 on the same subnet using 'sudo ip addr add 192.168.10.2/24 dev enp0s3', placing both VMs on the same virtual network.

```

sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0f:36:1b brd ff:ff:ff:ff:ff:ff
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo ip addr add 192.168.10.2/24 dev enp0s3
[sudo] password for sk-naser-hussain:
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$

```

Fig 3. Assigning static IP address 192.168.10.2/24 to VM2 (enp0s3).

Step 3: Test Connectivity Using the Ping Command

With both VMs configured on the 192.168.10.0/24 subnet, connectivity was verified in both directions using the ping command. Pinging 192.168.10.1 (VM1) returned 4 packets transmitted and 4 received with 0% packet loss, confirming successful connectivity.

```

sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ping -c 4 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=3.69 ms
64 bytes from 192.168.10.1: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 192.168.10.1: icmp_seq=3 ttl=64 time=0.097 ms
64 bytes from 192.168.10.1: icmp_seq=4 ttl=64 time=0.129 ms

--- 192.168.10.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3088ms
rtt min/avg/max/mdev = 0.066/0.995/3.688/1.554 ms
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$

```

Fig 4. Successful ping to VM1 (192.168.10.1) — 4 packets transmitted, 0% packet loss.

Similarly, pinging 192.168.10.2 (VM2) also returned 4 packets transmitted and 4 received with 0% packet loss, confirming bidirectional connectivity between the two virtual machines.

```

sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=1.66 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=0.054 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=0.061 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.067 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3101ms
rtt min/avg/max/mdev = 0.054/0.459/1.657/0.691 ms
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$

```

Fig 5. Successful ping to VM2 (192.168.10.2) — 4 packets transmitted, 0% packet loss.

Step 4: Designate VM1 as the Server and Create the TCP Server Program

VM1 (IP address 192.168.10.1) was designated as the Server. A Python TCP server program, server.py, was written using the socket module. The server binds to all available interfaces on port 5000, listens for an incoming connection, accepts the client's connection, and then enters a loop to receive a message from the client and send a reply, until either side sends 'bye'.

```
import socket
HOST="0.0.0.0"
PORT=5000
server_socket=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind((HOST,PORT))
server_socket.listen(1)
print("TCP server started.")
print("Waiting for client...")
conn,addr=server_socket.accept()
print("Connected by:",addr)
while True:
    data=conn.recv(1024).decode()
    if not data or data.lower()=="bye":
        break
    print("Client:",data)
    message=input("Server: ")
    conn.send(message.encode())
    if message.lower() == "bye":
        break
conn.close()
server_socket.close()
print("Server closed.")
```

A screenshot of a terminal window titled 'sk-naser-hussain@sk-naser-hussain-VirtualBox: ~'. The terminal shows the GNU nano 7.2 editor with a file named server.py. The code in the editor is the same Python TCP server program as shown in the previous block, with syntax highlighting. The code includes imports, host/port definitions, socket creation, binding, listening, accepting a connection, and a loop for receiving and sending data until 'bye' is received.

Fig 6. server.py — TCP server program created on VM1 using GNU nano.

Step 5: Designate VM2 as the Client and Create the TCP Client Program

VM2 (IP address 192.168.10.2) was designated as the Client. A corresponding Python TCP client program, client.py, was written. The client connects to the server's IP address (192.168.10.1) on port 5000, and then enters a loop to send a message to the server and receive a reply, until either side sends 'bye'.

```
import socket
HOST="192.168.10.1"
PORT=5000
```

```

client_socket=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((HOST,PORT))
print("Connected to server.")
while True:
    message=input("Client: ")
    client_socket.send(message.encode())
    if message.lower()=="bye":
        break
    data=client_socket.recv(1024).decode()
    if not data:
        break
    print("Server:",data)
    if data.lower()=="bye":
        break
client_socket.close()
print("Client closed.")

```



```

sk-naser-hussain@sk-naser-hussain-VirtualBox: ~
GNU nano 7.2 client.py
import socket
HOST="192.168.10.1"
PORT=5000
client_socket=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((HOST,PORT))
print("Connected to server.")
while True:
    message=input("Client: ")
    client_socket.send(message.encode())
    if message.lower()=="bye":
        break
    data=client_socket.recv(1024).decode()
    if not data:
        break
    print("Server:",data)
    if data.lower()=="bye":
        break
client_socket.close()
print("Client closed.")

```

Fig 7. client.py — TCP client program created on VM2 using GNU nano.

Step 6: Establish Connection and Demonstrate the Chat Application

On VM1, the server program was started using 'python3 server.py'. The server printed 'TCP server started.' and 'Waiting for client...', and upon the client connecting, displayed the client's IP address and port: Connected by: ('192.168.10.2', 41330). Messages were then exchanged interactively between the Server and the Client, with each message and its sender clearly logged in the terminal, until the session was ended and the server closed.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c8:c6:c8 brd ff:ff:ff:ff:ff:ff
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo ip addr add 192.168.10.1/24 dev enp0s3
[sudo] password for sk-naser-hussain:
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  server.py  snap  Templates  Videos
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ nano server.py
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ python3 server.py
TCP server started.
Waiting for client...
Connected by: ('192.168.10.2', 41330)
Client: Hello server!!
Server: Hello Client!!
Client: How are you?
Server: I am fine.
Server closed.
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig 8. VM1 (Server) — server started, client connected, chat messages exchanged, server closed.

On VM2, the client program was started using 'python3 client.py', which immediately connected to the server and printed 'Connected to server.'. The client and server then exchanged messages in real time ('Hello server!!' / 'Hello Client!!' and 'How are you?' / 'I am fine.'), after which the client sent 'bye' to gracefully terminate the session, and the client program closed the connection.

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0f:36:1b brd ff:ff:ff:ff:ff:ff
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo ip addr add 192.168.10.2/24 dev enp0s3
[sudo] password for sk-naser-hussain:
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ ls
client.py  Desktop  Documents  Downloads  Music  Pictures  Public  snap  Templates  Videos
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ nano clien.py
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ nano client.py
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ python3 client.py
Connected to server.
Client: Hello server!!
Server: Hello Client!!
Client: How are you?
Server: I am fine.
Client: bye
Client closed.
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig 9. VM2 (Client) — connected to server, chat messages exchanged, client closed.

Result / Conclusion

Two Ubuntu Virtual Machines were successfully configured on the same virtual network (192.168.10.0/24), with connectivity verified using the ping command in both directions. A TCP-based client-server chat application was implemented in Python, with VM1 (192.168.10.1) acting as the Server and VM2 (192.168.10.2) acting as the Client. The server successfully accepted a connection from the client, and messages were exchanged bidirectionally in real time until the session was closed gracefully by both ends. Screenshots of the network configuration, connectivity tests, program code, and chat communication were captured at every stage, confirming that all the stated objectives of the experiment were achieved.